Análise de Speedup

Busca de nomes em arquivos .txt com e sem paralelismo (Threads em Java)

1. Os dois sistemas

	Sem paralelismo	Com paralelismo
Fluxo	Lê todos os arquivos .txt, junta as linhas em uma única lista, ordena essa lista com Merge Sort e faz a busca linear nela.	Lê todos os arquivos .txt mantendo **uma lista por arquivo**, faz a busca linear em paralelo, concatena os resultados e ordena com Merge Sort apenas as linhas encontradas.
Threads	Busca sequencial.	Uma `SearchThread` (`extends Thread`) por arquivo: 5 no dataset P e 7 no G. A thread principal faz `join()` em cada uma e concatena os resultados.
Sincronização	Não se aplica.	Não é necessária. Cada thread grava apenas no próprio resultado, e o `join()` garante que a thread principal enxerga o que foi gravado.
Saída	Nome do arquivo, número da linha e texto da linha. As duas versões devolvem exatamente as mesmas linhas, o que foi conferido nos 8 casos testados.	

2. Metodologia

- **Datasets:** P tem 5 arquivos de 999 linhas (4.995 no total) e G tem 7 arquivos de 9.999 linhas (69.993 no total).
- **Nomes pesquisados:** "maria" (17 ocorrências no P e 284 no G) e "john" (91 no P e 1.395 no G).
- **Medição:** método `SearchUtils.Search` de cada sistema, o mesmo que a interface chama ao clicar em PESQUISAR (sem o tempo de preencher a tabela da tela). Cada configuração roda em uma JVM nova, com uma execução fria, seguida de 30 execuções de aquecimento (descartadas, para que o JIT otimize o código) e de 100 execuções medidas. O valor usado é a mediana das 100. Tudo foi repetido 3 vezes, em 3 JVMs, e o relatório usa a mediana das 3 medianas.
- **Métricas:** Speedup **S = T_{sem} / T_{com}** e Eficiência **E = S / p**, em que p é o número de núcleos usados (o menor valor entre o número de threads e os 6 núcleos disponíveis).

3. Resultados

Tabela 1. Sistema completo (tempo em ms, mediana)

Dataset	Nome	Ocorrências	Threads	T sem paralelismo	T com paralelismo	Speedup	Eficiência
P	maria	17	5	4,81	2,56	**1,88**	0,38
P	john	91	5	5,09	3,31	**1,54**	0,31
G	maria	284	7	83,37	11,20	**7,44**	1,24
G	john	1.395	7	82,34	11,66	**7,06**	1,18

Tabela 2. Primeira execução, com a JVM recém-iniciada (equivale ao primeiro clique em PESQUISAR)

Dataset	Nome	T sem (ms)	T com (ms)	Speedup
P	maria	59,5	37,9	**1,57**
P	john	57,8	43,5	**1,33**
G	maria	324,3	153,4	**2,11**
G	john	267,8	140,5	**1,91**

Figura 1. Onde o tempo é gasto (dataset G, nome "maria")

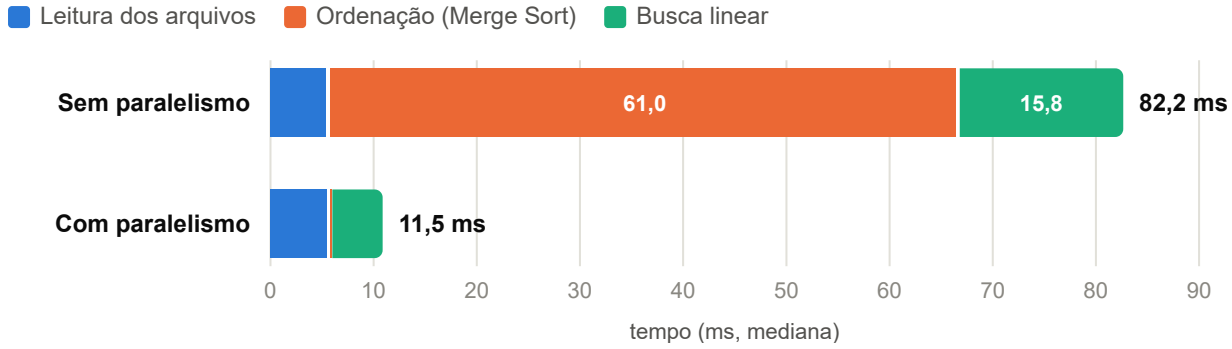


Tabela 3. Tempo por fase (ms, mediana, nome "maria")

Dataset	Sistema	Leitura	Ordenação	Busca	Total
P	Sem paralelismo	1,42	2,80	0,69	**5,21**
P	Com paralelismo	1,47	0,03	1,17	**2,71**
G	Sem paralelismo	5,77	61,03	15,85	**82,22**
G	Com paralelismo	5,84	0,15	4,87	**11,46**

Na versão com paralelismo, a coluna Busca corresponde à busca com threads e a coluna Ordenação corresponde à ordenação apenas das linhas encontradas.

Figura 2. Speedup apenas das threads (busca isolada, com os arquivos já em memória, nome "maria")

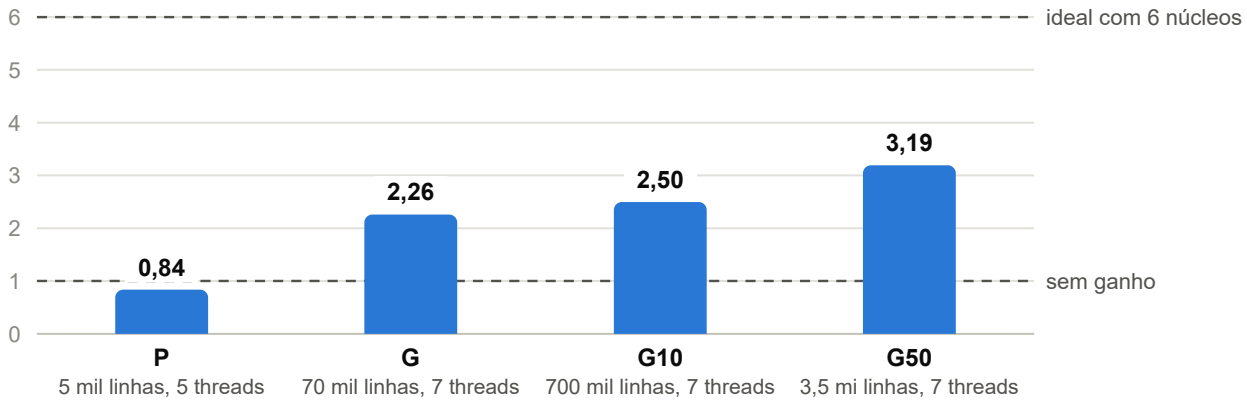


Tabela 4. Busca sequencial e busca com threads sobre a mesma entrada (ms, mediana)

Volume	Linhas	Threads	T sequencial	T com threads	Speedup	Eficiência
P	4.995	5	0,74	0,88	**0,84**	0,17
G	69.993	7	10,65	4,71	**2,26**	0,38
G10	699.930	7	98,40	39,40	**2,50**	0,42
G50	3.499.650	7	517,59	162,11	**3,19**	0,53

G10 e G50: cada arquivo do G repetido 10 e 50 vezes em memória, para medir como o ganho muda com mais dados.

4. Análise dos resultados

1. **O speedup de 7,4 no G não vem apenas das threads.** A eficiência passa de 1 (1,24), o que seria impossível usando só 6 núcleos. A diferença está no algoritmo: a versão com paralelismo ordena apenas as 284 linhas encontradas, enquanto a versão sem paralelismo ordena todas as 69.993 linhas. Essa ordenação consome 61,0 dos 82,2 ms (74% do tempo).
2. **Isolado, o ganho das threads é de 2,26 no G.** No P a versão com threads fica *mais lenta* (speedup de 0,84): buscar em 5 mil linhas leva cerca de 0,7 ms, a mesma ordem de grandeza do custo de criar, iniciar e esperar 5 threads.

3. **O speedup cresce com o volume de dados:** 2,26 no G e 3,19 no G50, porque o custo fixo de criar as threads pesa cada vez menos. Mesmo assim, fica longe do ideal de 6. Causas prováveis: o `split(" ")` cria muitos objetos, o que sobrecarrega o coletor de lixo e a memória compartilhada; são 7 threads para 6 núcleos; e o processador reduz o clock quando todos os núcleos estão ativos.
4. **Lei de Amdahl:** na versão sem paralelismo, a busca representa apenas 19% do tempo (15,8 de 82,2 ms). Se apenas a busca fosse paralelizada, mantendo a ordenação de todas as linhas, o speedup máximo seria 1,19 com 6 núcleos, e 1,24 mesmo com infinitos núcleos. O ganho grande veio de retirar trabalho sequencial do caminho.
5. **Primeira execução:** com a JVM fria, o speedup no G cai para 2,1 (de 324 para 153 ms), porque o código ainda não foi otimizado pelo JIT. É algo próximo desse tempo que a interface mostra no primeiro clique. Nas pesquisas seguintes, o tempo cai.

5. Conclusão

Com uma thread por arquivo, a busca no dataset G ficou 2,26 vezes mais rápida, e o sistema completo ficou 7,4 vezes mais rápido. A maior parte desse ganho veio de não ordenar todas as linhas. No dataset P, as threads não compensam, porque o custo de criá-las é maior que o trabalho de cada uma. O ganho das threads cresce com o volume de dados, chegando a 3,19 vezes no G50.